

CapyTown — Semana 11 · RC-2

“Las 3 Vueltas del Jirón”

Lane Following 1.0 — Visión HSV, IPM y Control PID

Universidad ESAN · Robótica 2026-I · Sección B

Prof. Marks Calderón Niquin

Reto Clasificatorio 2 · 4 pts · Escenario A “El Tambo” (16 tiles, 2.0 × 2.0 m)

El chaski aprende a correr por el jirón sin salirse al borde — respetando la línea blanca a la derecha y el eje amarillo al centro, como la pintura en cualquier carretera andina.

1. Objetivos de la sesión

Al terminar la semana 11 cada grupo debe ser capaz de:

1. Segmentar por color (HSV) el borde blanco del carril y el eje amarillo central en la imagen de la cámara del Yahboom Pi5.
2. Convertir la posición de las líneas en un **error lateral** (`/lane_error`, en metros) mediante una transformada de perspectiva inversa (IPM).
3. Cerrar el lazo con un **controlador PID** que publica `/cmd_vel` para mantener el robot centrado en el jirón.
4. Completar **3 vueltas autónomas** al Escenario A a ≥ 0.2 m/s y demostrarlo con video + plot de `/lane_error`.

2. Activación cognitiva (5 min, antes de tocar el teclado)

Cada grupo dibuja **en papel**, antes de abrir el código:

- ¿Qué le pasa a la trayectoria cuando **Kp es demasiado alto**? ¿Y demasiado bajo?
- ¿Qué forma esperas ver en la gráfica de `/lane_error` vs. tiempo en una recta? ¿Y en una curva?

La predicción se fotografía y se adjunta al informe. **La gráfica real será el árbitro.**

3. Teoría (1 h)

3.1 Repaso de segmentación HSV (semana 9)

El espacio HSV separa **tono** (H), **saturación** (S) y **valor/brillo** (V). Sobre fondo negro mate, el blanco tiene S bajo y V alto; el amarillo tiene $H \approx 20-35^\circ$, S y V altos. Por eso segmentamos por rango HSV y no por RGB: HSV es robusto a cambios de iluminación.

3.2 Transformada de perspectiva inversa (IPM)

La cámara ve el piso “en perspectiva”: líneas paralelas parecen converger. La **IPM** (Inverse Perspective Mapping) re proyecta la imagen a una **vista de pájaro** (bird’s-eye view) donde las distancias en píxeles son proporcionales a distancias reales en el piso. Esto permite convertir el desplazamiento lateral en píxeles a **metros**, que es lo que el PID necesita.

La IPM usa una homografía H (matriz 3×3) calculada con `cv2.getPerspectiveTransform` a partir de 4 puntos del piso de coordenadas conocidas (las esquinas de un tile, por ejemplo).

3.3 Control PID sobre el error lateral

El error lateral $e(t)$ = distancia (m) entre el centro del robot y el centro del carril. El PID calcula la velocidad angular:

$$\omega(t) = K_p \cdot e(t) + K_i \cdot \int e(t) dt + K_d \cdot de(t)/dt$$

- **Kp (proporcional):** corrige proporcional al error actual. Demasiado alto → oscila.
- **Ki (integral):** elimina el error sostenido (sesgo), p. ej. en curvas largas. Demasiado alto → sobreoscilación/wind-up.
- **Kd (derivativo):** amortigua, anticipa el cambio. Reduce oscilación pero amplifica ruido.

look_ahead_distance define a qué altura del piso (cuántos metros adelante) medimos el error — un look-ahead mayor suaviza pero reacciona más tarde.

Regla de oro del lab: *sintonizar un parámetro a la vez (primero Kp, luego Kd, al final Ki si hace falta), comparando las curvas antes/después de cada ajuste y guardando un ros2 bag por cada cambio.*

4. Arquitectura de nodos de la Semana 11



Tópico	Tipo	Publicado por	Consumido por
/camera/image_raw	sensor_msgs/Image	cámara (bringup)	lane_detector
/lane_error	std_msgs/Float32	lane_detector	lane_controller
/lane/debug_image	sensor_msgs/Image	lane_detector	RViz (debug)
/cmd_vel	geometry_msgs/Twist	lane_controller	base de motores

Convención de signo de /lane_error: positivo = el centro del carril está a la **derecha** del robot (robot desviado a la izquierda) → el controlador debe girar a la derecha ($\omega < 0$). Mantener este convenio en todo el grupo.

5. Código base

Paquete: capytown_esan_pkg/capytown_esan/. Código probado contra **ROS2 Humble + rclpy + cv_bridge + OpenCV**. Los bloques # todo son lo que cada grupo debe completar.

5.1 config/hsv_params.yaml

```
# CapyTown - Rangos HSV calibrados para el dia del lab.
# Recalibrar SIEMPRE con scripts/hsv_tuner.py al inicio de la sesion.
lane_detector:
  ros__parameters:
    # Blanco - borde derecho del carril
    white_h_min: 0
    white_h_max: 180
    white_s_min: 0
    white_s_max: 30
    white_v_min: 180
    white_v_max: 255
    # Amarillo - eje central discontinuo
    yellow_h_min: 20
    yellow_h_max: 35
    yellow_s_min: 100
    yellow_s_max: 255
    yellow_v_min: 100
    yellow_v_max: 255
    # Geometria / IPM
    min_area: 150          # area minima de contorno (px) para filtrar ruido
    lane_width_m: 0.21     # ancho de carril CappyTown (m)
    px_per_meter: 600.0    # escala de la vista IPM (px por metro) - calibrar
    look_ahead_row: 0.6    # fila de muestreo: 0=arriba, 1=abajo de la imagen IPM
    publish_debug: true
```

5.2 config/pid_params.yaml

```
lane_controller:
  ros__parameters:
    kp: 2.5
    ki: 0.0
    kd: 0.3
    linear_speed: 0.20    # m/s - velocidad de cruce (min. aceptable 0.15)
    max_angular: 2.0      # rad/s - saturacion de omega
    integral_limit: 0.5   # anti-windup: limite del termino integral
    error_timeout: 0.5    # s sin /lane_error => frena por seguridad
    control_rate: 30.0    # Hz del lazo de control
```

5.3 capytown_esan/lane_detector.py

□ **TODO 1 (paso 4 del nodo):** calcular el centro del carril en píxeles y el error lateral en metros, manejando los casos en que solo se ve una de las dos líneas.

```
#!/usr/bin/env python3
"""CapyTown lane_detector - Semana 11 (RC-2).

Segmenta el borde blanco y el eje amarillo por HSV, aplica una vista de
pajaro (IPM) y publica el error lateral en metros sobre /lane_error.
"""
import cv2
import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import Float32
from cv_bridge import CvBridge

class LaneDetector(Node):
    def __init__(self):
        super().__init__('lane_detector')
        self.bridge = CvBridge()

    # --- Declaracion de parametros (cargados desde hsv_params.yaml) ---
    self.declare_parameters('', [
        ('white_h_min', 0), ('white_h_max', 180),
        ('white_s_min', 0), ('white_s_max', 30),
        ('white_v_min', 180), ('white_v_max', 255),
        ('yellow_h_min', 20), ('yellow_h_max', 35),
        ('yellow_s_min', 100), ('yellow_s_max', 255),
        ('yellow_v_min', 100), ('yellow_v_max', 255),
        ('min_area', 150),
```

```

        ('lane_width_m', 0.21),
        ('px_per_meter', 600.0),
        ('look_ahead_row', 0.6),
        ('publish_debug', True),
    ])
    gp = self.get_parameter
    self.white_lo = np.array([gp('white_h_min').value,
                             gp('white_s_min').value,
                             gp('white_v_min').value])
    self.white_hi = np.array([gp('white_h_max').value,
                             gp('white_s_max').value,
                             gp('white_v_max').value])
    self.yellow_lo = np.array([gp('yellow_h_min').value,
                              gp('yellow_s_min').value,
                              gp('yellow_v_min').value])
    self.yellow_hi = np.array([gp('yellow_h_max').value,
                              gp('yellow_s_max').value,
                              gp('yellow_v_max').value])
    self.min_area = gp('min_area').value
    self.lane_width_m = gp('lane_width_m').value
    self.px_per_meter = gp('px_per_meter').value
    self.look_ahead_row = gp('look_ahead_row').value
    self.publish_debug = gp('publish_debug').value

    self.M = None          # homografia IPM (se calcula al primer frame)
    self.warp_size = None

    self.sub = self.create_subscription(
        Image, '/camera/image_raw', self.on_image, 10)
    self.pub_err = self.create_publisher(Float32, '/lane_error', 10)
    self.pub_dbg = self.create_publisher(Image, '/lane/debug_image', 10)
    self.get_logger().info('lane_detector listo.')

# --- IPM: homografia de vista frontal a vista de pajarito ---
def build_ipm(self, w, h):
    # 4 puntos del trapecio del piso en la imagen original (ajustar al lab)
    src = np.float32([[0.18 * w, 0.62 * h], [0.82 * w, 0.62 * h],
                     [1.00 * w, 0.98 * h], [0.00 * w, 0.98 * h]])
    dst = np.float32([[0.30 * w, 0.0], [0.70 * w, 0.0],
                     [0.70 * w, h], [0.30 * w, h]])
    self.M = cv2.getPerspectiveTransform(src, dst)
    self.warp_size = (w, h)

def on_image(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    h, w = frame.shape[:2]
    if self.M is None:
        self.build_ipm(w, h)

    # 1) Vista de pajarito (IPM)
    warp = cv2.warpPerspective(frame, self.M, self.warp_size)
    hsv = cv2.cvtColor(warp, cv2.COLOR_BGR2HSV)

    # 2) Mascaras de color
    mask_white = cv2.inRange(hsv, self.white_lo, self.white_hi)
    mask_yellow = cv2.inRange(hsv, self.yellow_lo, self.yellow_hi)
    kernel = np.ones((3, 3), np.uint8)
    mask_white = cv2.morphologyEx(mask_white, cv2.MORPH_OPEN, kernel)
    mask_yellow = cv2.morphologyEx(mask_yellow, cv2.MORPH_OPEN, kernel)

    # 3) Fila de muestreo (look-ahead)
    row = int(self.look_ahead_row * h)
    band = slice(max(0, row - 8), min(h, row + 8))

    x_white = self._centroid_x(mask_white[band, :])
    x_yellow = self._centroid_x(mask_yellow[band, :])

    # 4) Centro del carril (px) -> error lateral (m)
    # TODO (alumno): calcular el centro del carril y el error en metros.
    # - Si se ven ambas lineas: centro = (x_white + x_yellow) / 2
    # - Si solo se ve una: estimar usando lane_width_m * px_per_meter
    # - error_m = (centro - w/2) / px_per_meter (signo segun convenio)

```

```

        center_px = None          # <-- completar
        error_m = 0.0             # <-- completar

        out = Float32()
        out.data = float(error_m)
        self.pub_err.publish(out)

        if self.publish_debug:
            self._publish_debug(warp, row, x_white, x_yellow, center_px, msg)

    @staticmethod
    def _centroid_x(mask):
        m = cv2.moments(mask, binaryImage=True)
        if m['m00'] < 1e-3:
            return None
        return m['m10'] / m['m00']

    def _publish_debug(self, warp, row, xw, xy, xc, header_msg):
        dbg = warp.copy()
        cv2.line(dbg, (0, row), (dbg.shape[1], row), (0, 255, 0), 1)
        for x, color in ((xw, (255, 255, 255)), (xy, (0, 255, 255)),
                        (xc, (0, 0, 255))):
            if x is not None:
                cv2.circle(dbg, (int(x), row), 5, color, -1)
        out = self.bridge.cv2_to_imgmsg(dbg, 'bgr8')
        out.header = header_msg.header
        self.pub_dbg.publish(out)

def main(args=None):
    rclpy.init(args=args)
    node = LaneDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

5.4 capytown_esan/lane_controller.py

□ **TODO 2 (lazo de control):** implementar los términos P, I (con anti-windup) y D, y saturar la velocidad angular.

```

#!/usr/bin/env python3
"""CapyTown lane_controller - Semana 11 (RC-2).

Controlador PID sobre /lane_error que publica /cmd_vel.
"""
import math
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
from geometry_msgs.msg import Twist

class LaneController(Node):
    def __init__(self):
        super().__init__('lane_controller')
        self.declare_parameters('', [
            ('kp', 2.5), ('ki', 0.0), ('kd', 0.3),
            ('linear_speed', 0.20),
            ('max_angular', 2.0),
            ('integral_limit', 0.5),
            ('error_timeout', 0.5),

```

```

        ('control_rate', 30.0),
    ])
    gp = self.get_parameter
    self.kp = gp('kp').value
    self.ki = gp('ki').value
    self.kd = gp('kd').value
    self.v = gp('linear_speed').value
    self.max_w = gp('max_angular').value
    self.i_limit = gp('integral_limit').value
    self.timeout = gp('error_timeout').value
    rate = gp('control_rate').value

    self.error = None
    self.last_error = 0.0
    self.integral = 0.0
    self.last_stamp = self.get_clock().now()
    self.last_rx = self.get_clock().now()

    self.sub = self.create_subscription(
        Float32, '/lane_error', self.on_error, 10)
    self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
    self.timer = self.create_timer(1.0 / rate, self.control_loop)
    self.get_logger().info('lane_controller listo.')

def on_error(self, msg):
    if not math.isnan(msg.data):
        self.error = msg.data
        self.last_rx = self.get_clock().now()

def control_loop(self):
    now = self.get_clock().now()
    dt = (now - self.last_stamp).nanoseconds * 1e-9
    self.last_stamp = now
    if dt <= 0.0:
        return

    # Seguridad: sin error reciente => frenar
    age = (now - self.last_rx).nanoseconds * 1e-9
    if self.error is None or age > self.timeout:
        self.pub.publish(Twist()) # v=0, w=0
        self.integral = 0.0
        return

    # --- PID ---
    # TODO (alumno): implementar P, I (con anti-windup) y D.
    # P = kp * error
    # integral += error * dt ; clamp a [-i_limit, i_limit] ; I = ki * integral
    # derivative = (error - last_error) / dt ; D = kd * derivative
    # w = P + I + D ; saturar a [-max_w, max_w]
    w = 0.0 # <-- completar
    self.last_error = self.error

    cmd = Twist()
    cmd.linear.x = self.v
    cmd.angular.z = max(-self.max_w, min(self.max_w, w))
    self.pub.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = LaneController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.pub.publish(Twist()) # frena al salir
        node.destroy_node()
        rclpy.shutdown()

```

```
if __name__ == '__main__':
    main()
```

5.5 launch/lane_following.launch.py

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    pkg = get_package_share_directory('capytown_esan_pkg')
    hsv = os.path.join(pkg, 'config', 'hsv_params.yaml')
    pid = os.path.join(pkg, 'config', 'pid_params.yaml')
    return LaunchDescription([
        Node(package='capytown_esan_pkg', executable='lane_detector',
            name='lane_detector', parameters=[hsv], output='screen'),
        Node(package='capytown_esan_pkg', executable='lane_controller',
            name='lane_controller', parameters=[pid], output='screen'),
    ])
```

Recordar: registrar ambos nodos como entry_points en setup.py y declarar las dependencias en package.xml (rclpy, sensor_msgs, std_msgs, geometry_msgs, cv_bridge).

6. Procedimiento de laboratorio (2 h)

1. **Bringup** del robot: encender, SSH, exportar ROS_DOMAIN_ID del grupo, lanzar cámara.
2. **Calibrar HSV** el mismo día con scripts/hsv_tuner.py bajo la iluminación real del lab. Guardar los valores en hsv_params.yaml.
3. **Verificar IPM:** con publish_debug: true, abrir /lane/debug_image en RViz; las líneas deben verse rectas y verticales en la vista de pájaro. Ajustar los 4 puntos src si convergen.
4. **Lanzar** ros2 launch capytown_esan_pkg lane_following.launch.py.
5. **Sintonizar el PID** — un parámetro por prueba (sec. 3.3), grabando un ros2 bag por cada cambio:

```
ros2 bag record /lane_error /cmd_vel /odom -o s11_kp25
```

6. **Prueba cronometrada:** 3 vueltas consecutivas autónomas a ≥ 0.2 m/s. Cronometrar cada vuelta.
7. **Generar el plot** de /lane_error vs. tiempo desde el bag.

Snippet para el plot de /lane_error (entregable)

```
# scripts/plot_lane_error.py - uso: python3 plot_lane_error.py <ruta_bag>
import sys
import matplotlib.pyplot as plt
from rosbag2_py import SequentialReader, StorageOptions, ConverterOptions
from rclpy.serialization import deserialize_message
from std_msgs.msg import Float32

reader = SequentialReader()
reader.open(StorageOptions(uri=sys.argv[1], storage_id='sqlite3'),
    ConverterOptions('', ''))
t0, ts, errs = None, [], []
while reader.has_next():
    topic, data, stamp = reader.read_next()
    if topic == '/lane_error':
        t0 = stamp if t0 is None else t0
        ts.append((stamp - t0) * 1e-9)
        errs.append(deserialize_message(data, Float32).data)
plt.plot(ts, errs)
```

```
plt.axhline(0, ls='--', c='gray')
plt.xlabel('tiempo (s)'); plt.ylabel('/lane_error (m)')
plt.title('Error lateral - 3 vueltas RC-2'); plt.grid(True)
plt.savefig('lane_error_s11.png', dpi=150)
```

7. Entregable Git — capytown_G<n>_s11

- Nodos `lane_detector` y `lane_controller` con ganancias del PID configurables por YAML.
- `hsv_params.yaml` calibrado con los valores del día del lab.
- Video MP4: 3 vueltas completas sin intervención humana.
- Plot `lane_error_s11.png` de `/lane_error` vs. tiempo durante las 3 vueltas.
- Foto de la predicción de la activación cognitiva (sec. 2).
- *(Bonus)* Comparativa con/sin IPM.

Entregar con tag de git `s11` sobre la rama del grupo. El `ros2 bag` de la corrida final debe existir (no subirlo al repo por tamaño, pero el plot debe generarse a partir de él).

8. Rúbrica RC-2

El RC-2 se califica sobre **20 puntos: 60 % demostración + entregables (12 pts)** y **40 % defensa oral (8 pts)**. La defensa es individual: cualquier integrante puede ser interrogado sobre cualquier parte del sistema.

Conversión al curso: *el RC-2 aporta 4 pts al proyecto CapyTown → Nota RC-2 = (puntaje / 20) × 4. Ejemplo: 15/20 = 3.0 pts.*

8.1 Demostración y entregables — 60 % (12 pts)

Criterio	Puntaje
3 vueltas completas autónomas sin salirse del jirón (binario: ninguna rueda toca tile V)	4
Velocidad promedio ≥ 0.2 m/s en las 3 vueltas (medida del bag, no a ojo)	4
Error lateral medio $ lane_error \leq 3$ cm durante las vueltas + entregables completos (video, plot, YAML calibrado, foto de predicción)	4

8.2 Defensa oral — 40 % (8 pts)

Criterio (2 pts c/u)	Logrado (2)	En proceso (1)	Insuficiente (0)
Visión: explica la segmentación HSV, por qué no RGB, y cómo calibró los rangos del día	Explica con precisión y conecta con la evidencia de su robot	Explica parcialmente o solo recita el código	No puede explicar lo que su código hace
IPM: explica la homografía, la conversión píxeles → metros y cómo verificó la vista de pájaro	Explica el porqué de cada paso y su calibración	Confunde conceptos o no justifica la calibración	No distingue vista frontal de vista de pájaro
Control: justifica los valores finales de K_p , K_i , K_d con las curvas de $lane_error$ (bags antes/después)	Justifica cada ganancia con evidencia graficada	Justifica sin evidencia o cambió varios parámetros a la vez	Valores copiados sin justificación
Diseño: explica el convenio de signo, el grafo de nodos/tópicos y el comportamiento de seguridad (timeout/NaN)	Domina la arquitectura y sus decisiones	Conoce los nodos pero no las decisiones de diseño	No puede describir el flujo de datos

Bonus	Puntaje
-------	---------

Implementa IPM y el error lateral mejora ≥ 20 % respecto a la versión sin IPM (comparativa documentada)	+5 (tope 20/20)
--	-----------------

Impacto en Grand Prix: el tiempo promedio por vuelta de RC-2 determina la **posición de salida en la grilla del Grand Prix Ronda 1**. El grupo más rápido elige su slot. Además, RC-2 con puntaje completo (junto con RC-3 completo) otorga **+1 crédito en Ronda 1**.

9. Cierre reflexivo (5 min)

- ¿Por qué el mismo PID que funciona en la recta oscila en la curva?
- ¿Qué rol juega la velocidad en la estabilidad del controlador?
- ¿IPM mejoró el error o solo lo desplazó?
- ¿Qué hubiera pasado si cambiaran Kp y Kd al mismo tiempo sin registrar las curvas?

10. Errores comunes y diagnóstico

Síntoma	Causa probable	Acción
El robot zigzaguea	Kp demasiado alto	Bajar Kp 20–30 %; subir Kd.
Reacciona tarde, corta curvas	Kp bajo o look-ahead grande	Subir Kp; reducir look_ahead_row hacia abajo.
/lane_error salta a NaN	No detecta ninguna línea	Recalibrar HSV; revisar puntos src de la IPM.
Líneas curvas en debug_image	IPM mal calibrada	Reajustar los 4 puntos src sobre las esquinas de un tile.
El robot frena solo en recta	error_timeout muy corto o cámara lenta	Subir error_timeout; bajar resolución de cámara.
Deriva constante a un lado	Sesgo no corregido	Añadir Ki pequeño (con anti-windup).

CapyTown Tile Specification v1.0 — Edición Andina · Universidad ESAN · Robótica 2026-I
“Ama suwa, ama llulla, ama quella”. El chaski siempre cumple su misión.